



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





04

Technologia obiektowa & UML

Technologia obiektowa

Object Oriented
Analysis and
Design



Zestaw zasad tworzenia
oprogramowania wraz z
językami, bazami danych i
innymi narzędziami, które te
zasady wspomagają

Object Technology – A Manager's Guide, Taylor, 1997

OOAD & 6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa (komponenty)

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami

Object Oriented
Analysis and
Design



Siła technologii OOAD



Paradygmat jedności: jeden język stosowany przez wszystkich uczestników projektu



Ułatwia ponowne wykorzystanie: architektury i kodu



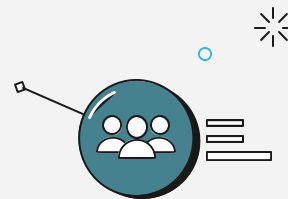
Model lepiej oddaje rzeczywistość



Stabilność



Adaptacja zmian



Technologia obiektowa

Abstrakcja

Hermetyzacja

Modularność

Hierarchia

Technologia obiektowa

Abstrakcja

- ✓ Istotna charakterystyka jednostki odróżniająca ją od wszystkich jednostek innego typu
- ✓ Definiuje pewien ograniczony obszar zależny od perspektywy
- ✓ Nie jest konkretnym przykładem, a jedynie oznacza istotę czegoś

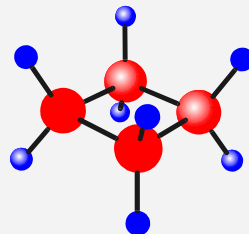
Technologia obiektowa

Abstrakcja

- ✓ Istotna charakterystyka jednostki odróżniająca ją od wszystkich jednostek innego typu
- ✓ Definiuje pewien ograniczony obszar zależny od perspektywy
- ✓ Nie jest konkretnym przykładem, a jedynie oznacza istotę czegoś



PROFESOR



LEKCJA CHEMII

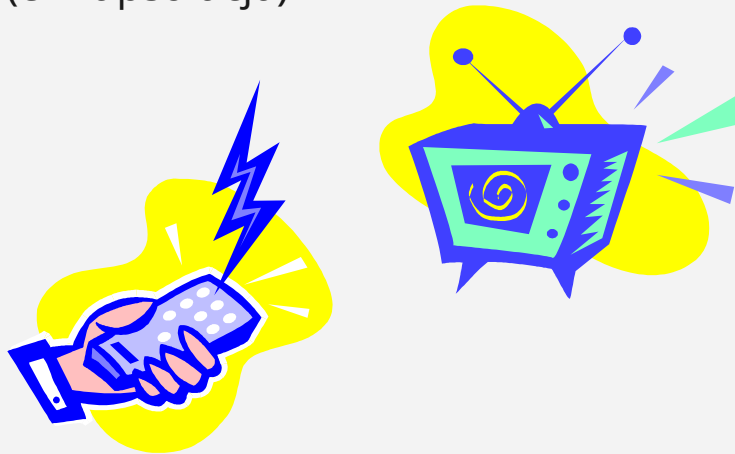


STUDENT

Technologia obiektowa

Hermetyzacja

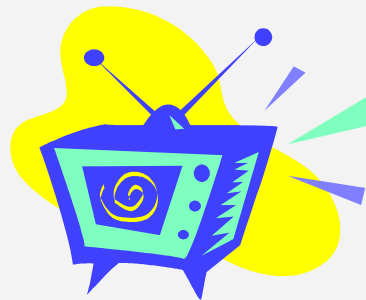
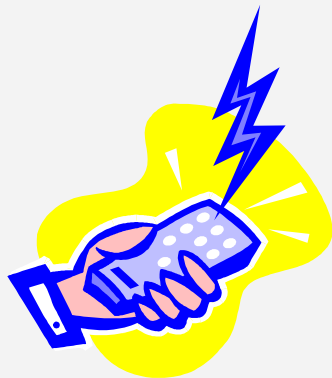
- ✓ Ukrywanie implementacji i korzystanie wyłącznie z interfejsów (enkapsulacja)



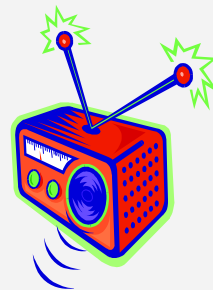
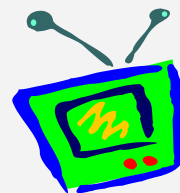
Technologia obiektowa

Hermetyzacja

- ✓ Ukrywanie implementacji i korzystanie wyłącznie z interfejsów (enkapsulacja)



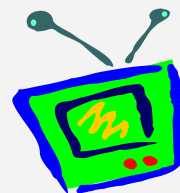
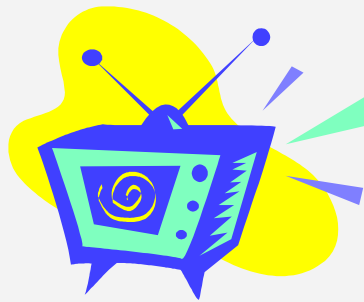
POLIMORFIZM



Technologia obiektowa

Hermetyzacja

- ✓ Ukrywanie implementacji i korzystanie wyłącznie z interfejsów (enkapsulacja)



POLIMORFIZM

Możliwość umieszczenia wielu różnych implementacji za jednym interfejsem

Technologia obiektowa

Modularność

- ✓ Podział całości na łatwe do zarządzania części
- ✓ Pomaga ogarnąć i zrozumieć skomplikowane systemy



Technologia obiektowa

Modularność

- ✓ Podział całości na łatwe do zarządzania części
- ✓ Pomaga ogarnąć i zrozumieć skomplikowane systemy



ZAPISY NA KURSY



OPŁATY



KALENDARZ



STUDENCI

Technologia obiektowa

Hierarchia

Wzrasta poziom abstrakcji

Majątek

Konta
bankowe

Środki
trwałe

Zabezpieczenia

Walutowe

Złotówkowe

Akcje

Weksle

Maleje poziom abstrakcji

Co to jest obiekt?

Reprezentuje pewną jednostkę
albo fizyczną, albo
konceptyjną, albo
oprogramowania

- ✓ Jednostka fizyczna
- ✓ Jednostka konceptyjna
- ✓ Jednostka oprogramowania

Co to jest obiekt?

Reprezentuje pewną jednostkę
albo fizyczną, albo
konceptyjną, albo
oprogramowania

✓ Jednostka fizyczna



samolot

✓ Jednostka konceptyjna

✓ Jednostka oprogramowania

Co to jest obiekt?

Reprezentuje pewną jednostkę
albo fizyczną, albo
konceptyjną, albo
oprogramowania

- ✓ Jednostka fizyczna



samolot

- ✓ Jednostka koncepcyjna



proces pobierania krwi

- ✓ Jednostka oprogramowania

Co to jest obiekt?

Reprezentuje pewną jednostkę
albo fizyczną, albo
konceptyjną, albo
oprogramowania

- ✓ Jednostka fizyczna



samolot

- ✓ Jednostka koncepcyjna



proces pobierania krwi

- ✓ Jednostka oprogramowania



lista

• Obiekt definicja •

Obiekt jest jednostką o dobrze zdefiniowanym zakresie, charakteryzującą się pewnym **stanem** i **zachowaniem**.

- ✓ **Stan:**
- ✓ **Zachowanie:** operacje, metody i automaty stanów

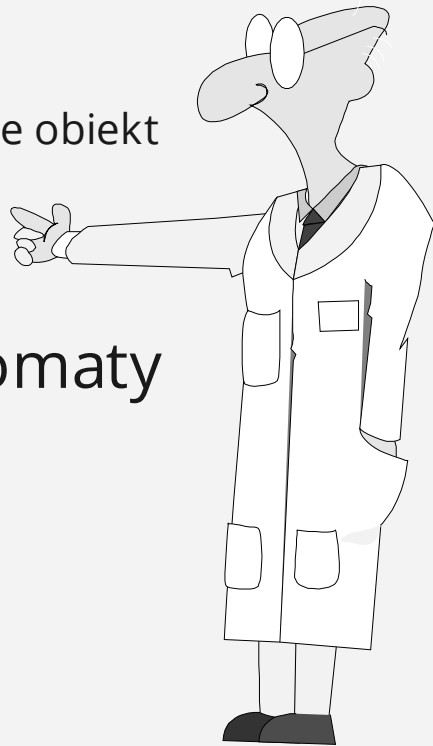
• Obiekt definicja •

- ✓ **Stan:** atrybuty i relacje

- ✓ Jedna z możliwych okoliczności w jakich istnieje obiekt
- ✓ Zmienia się w czasie

- ✓ **Zachowanie:** operacje, metody i automaty stanów

- ✓ Określa jak obiekt się zachowuje i jak reaguje
- ✓ Widoczne zachowanie jest modelowane przez wiadomości na które umie odpowiedzieć



• Obiekt jest unikalny •

Każdy obiekt jest **unikalną** jednostką nawet, jeżeli jest w takim samym stanie jak inny obiekt.



- ✓ **Maria Nowak**
- ✓ Uczy matematyki



- ✓ **Maria Nowak**
- ✓ Uczy matematyki

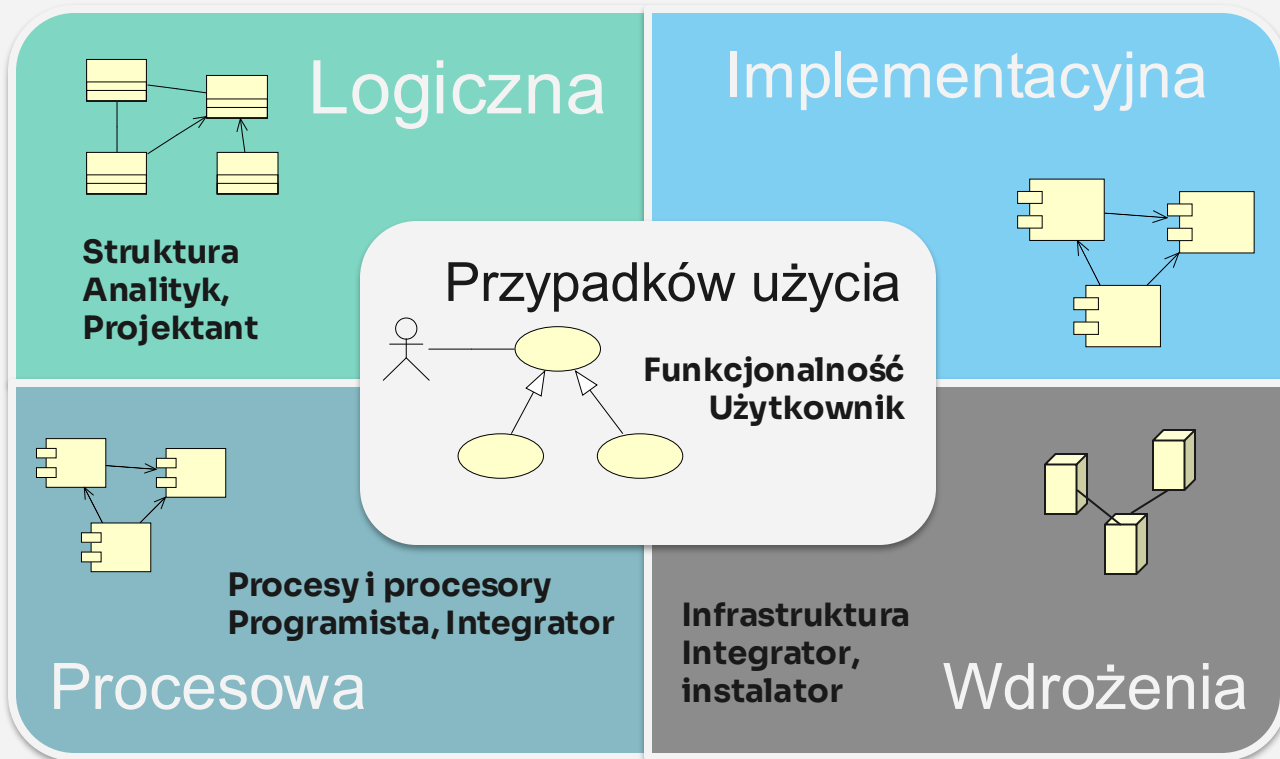
Co to jest klasa?

Klasa jest opisem zbioru obiektów posiadających ten sam zbiór atrybutów, operacji, relacji oraz semantykę.

Obiekt: jest instancją klasy

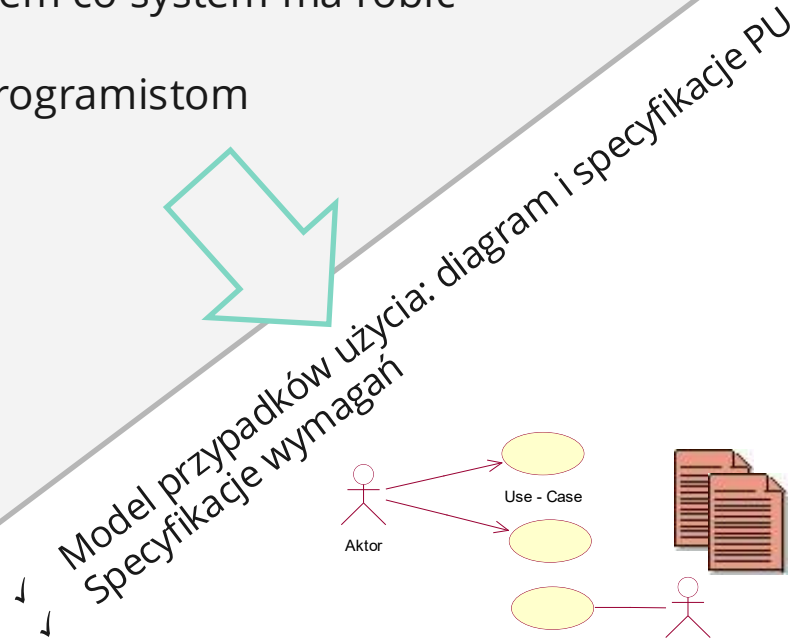
Klasa: jest abstrakcją; podkreśla pewną charakterystykę i ukrywa inne cechy

Architektura: 4+1 perspektywy



01 Zbieramy wymagania

-  Sprecyzowanie i uzgodnienie z klientem co system ma robić
-  Przekazanie właściwych informacji programistom
-  Ograniczenie systemu
-  Baza dla planu pracy
-  Baza dla oszacowania kosztów
-  Definicja interfejsu systemu



Zachowanie systemu



Określa jak system się zachowuje i jak reaguje



Jest widoczne na zewnątrz i możliwe do przetestowania



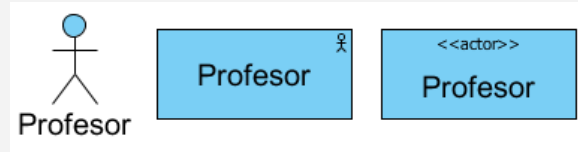
Jest opisane przez PRZYPADKI UŻYCIA (Use-Cases)



PU opisują:

system, jego środowisko i relacje pomiędzy systemem i środowiskiem

Zachowanie systemu: Aktor



Ktoś/coś kto/co wchodzi w interakcję z systemem z zewnątrz



Inicjuje akcje



Może reprezentować osobę fizyczną, rolę w systemie lub system zewnętrzny

• Zachowanie systemu: Przypadek użycia •



Sekwencja akcji wykonywanych przez system, których wynikiem jest konkretny (obserwowalny) rezultat mający znaczenie dla konkretnego aktora

Diagram Przypadków Użycia

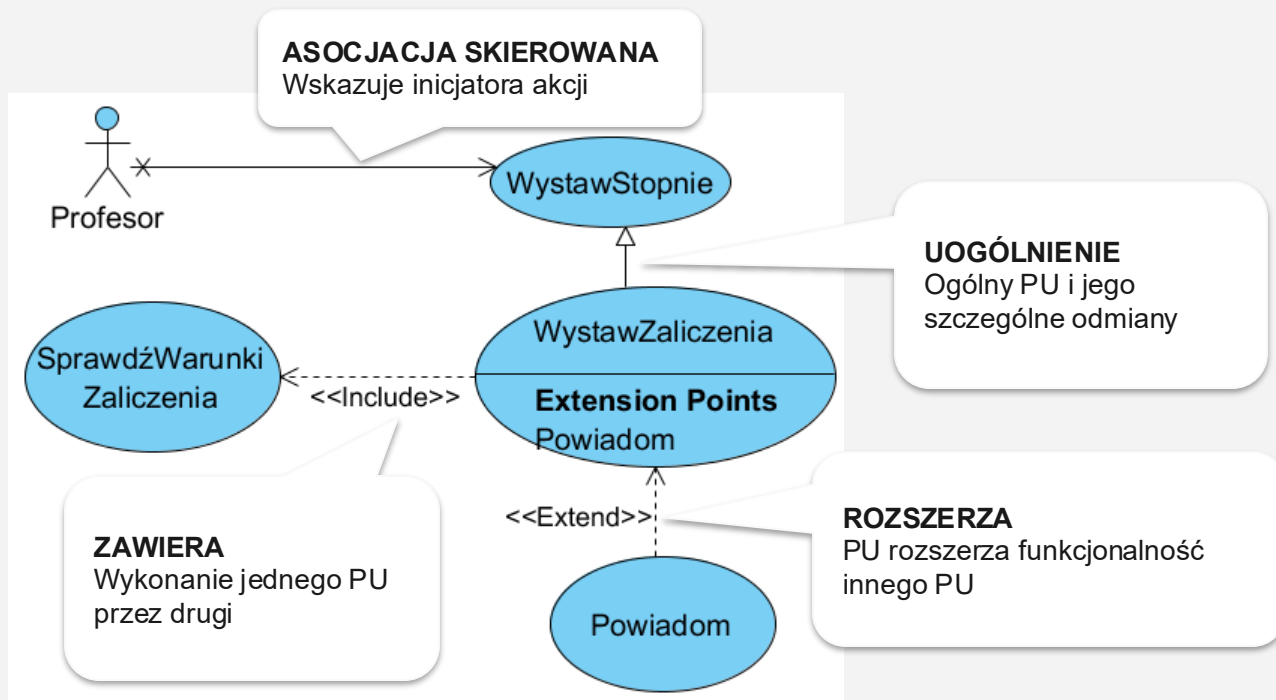


Diagram Przypadków Użycia

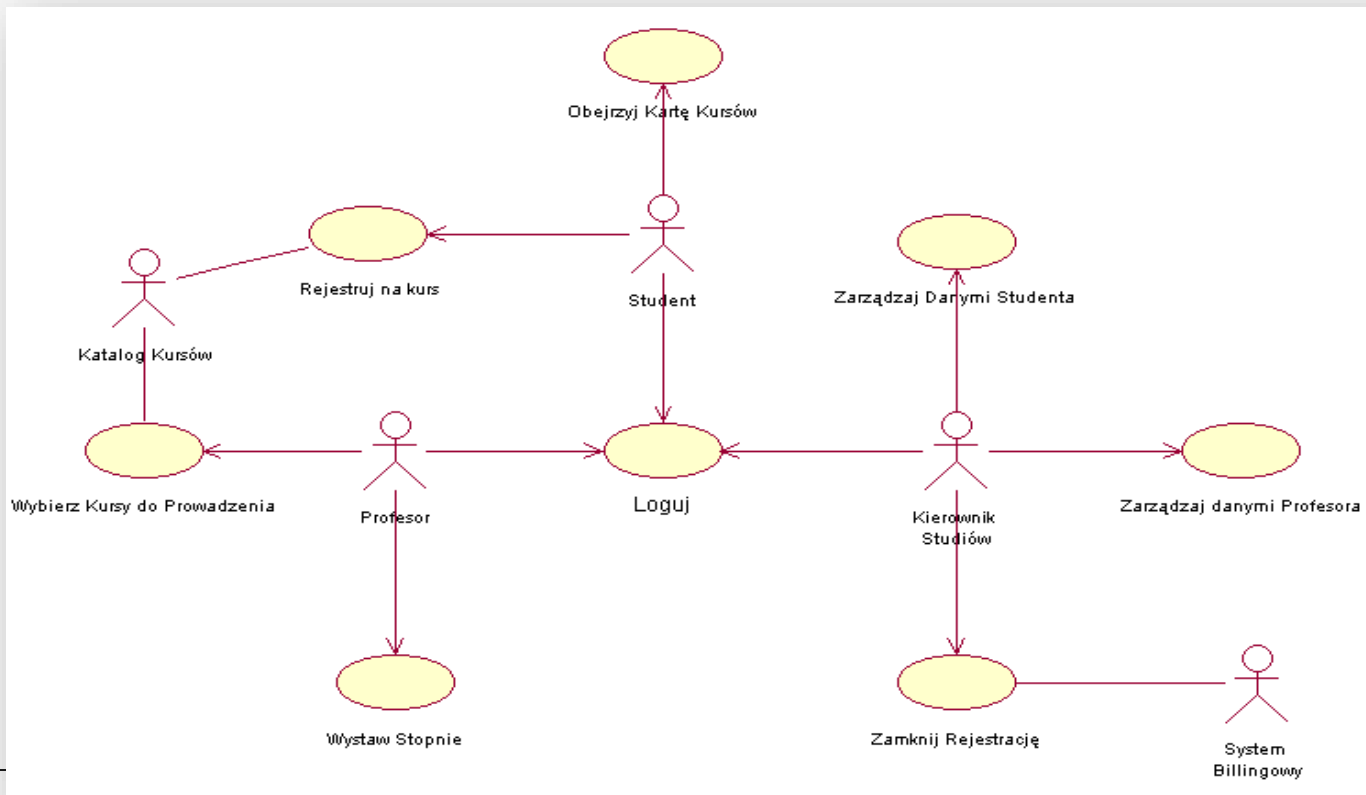
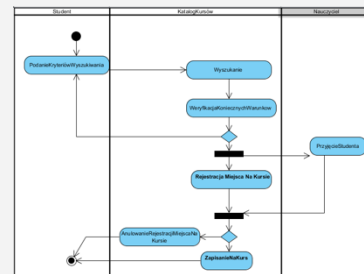


Diagram czynności

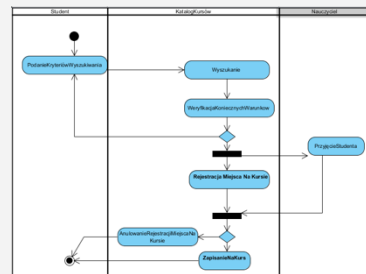


Reprezentuje przebieg zdarzeń od akcji do akcji oraz wynikające z nich zmiany stanów: deterministyczny automat skończenie stanowy



W modelu PU może być wykorzystany do przedstawienia kolejnych akcji w PU

Diagram czynności

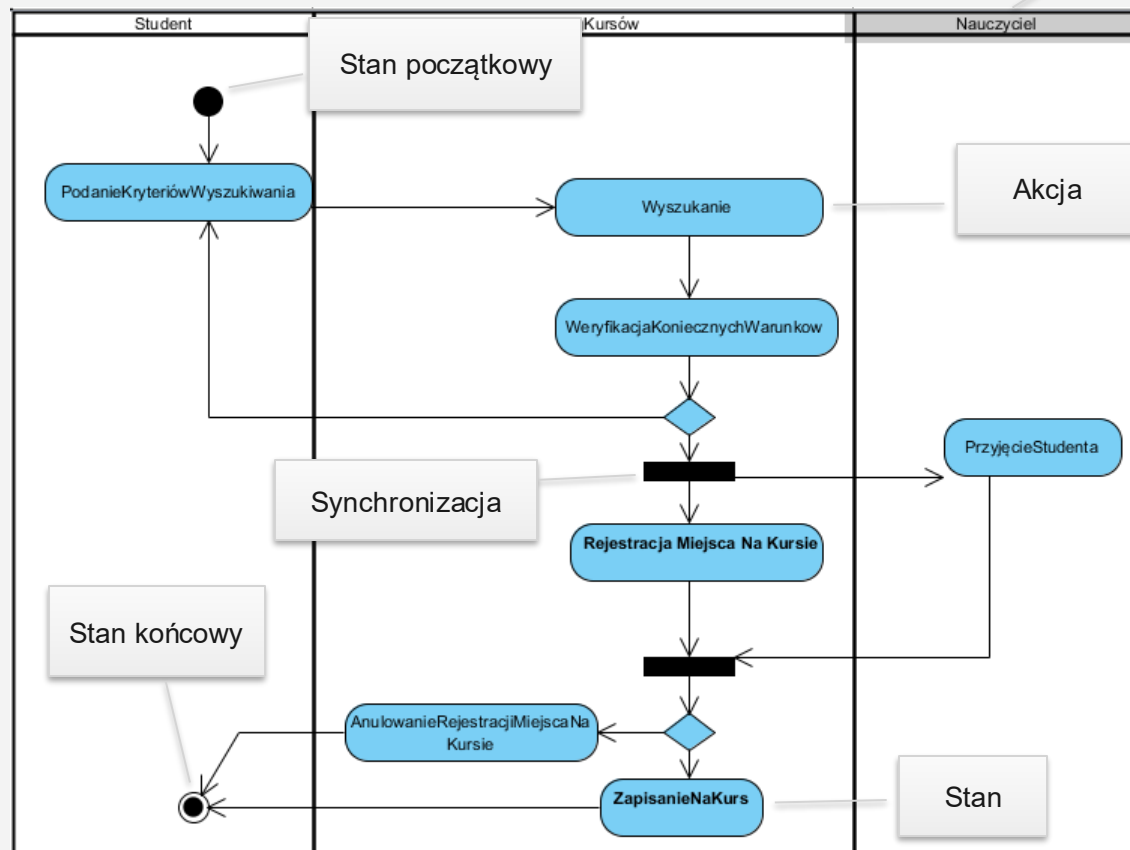


Reprezentuje przebieg zdarzeń od akcji do akcji oraz wynikające z nich zmiany stanów: deterministyczny automat skończenie stanowy



W modelu PU może być wykorzystany do przedstawienia kolejnych akcji w PU

Diagram czynności



Zalety modelu



Używany do komunikacji z klientami i ekspertami:

- ✓ Pozwala sprzedąć produkt na wczesnym etapie rozwoju
- ✓ Zapewnia jednolite zrozumienie wymagań



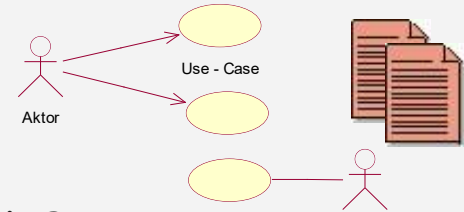
Używany do identyfikacji:

- ✓ Kto wchodzi w interakcję z systemem i jak ma reagować
- ✓ Interfejsów jakie powinien posiadać system



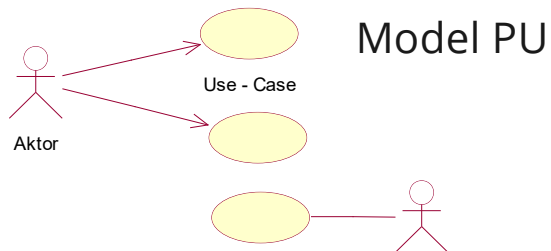
Używany do weryfikacji:

- ✓ Czy zostały wychwycone wszystkie wymagania?
- ✓ Czy zespół oprogramowania rozumie wymagania?



02 Analiza i projektowanie

01

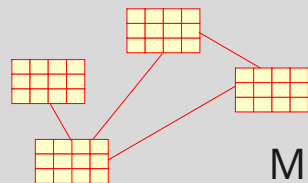


Specyfikacje wymagań

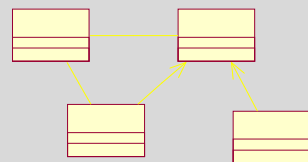
02



Opis architektury



Model danych



Projekt

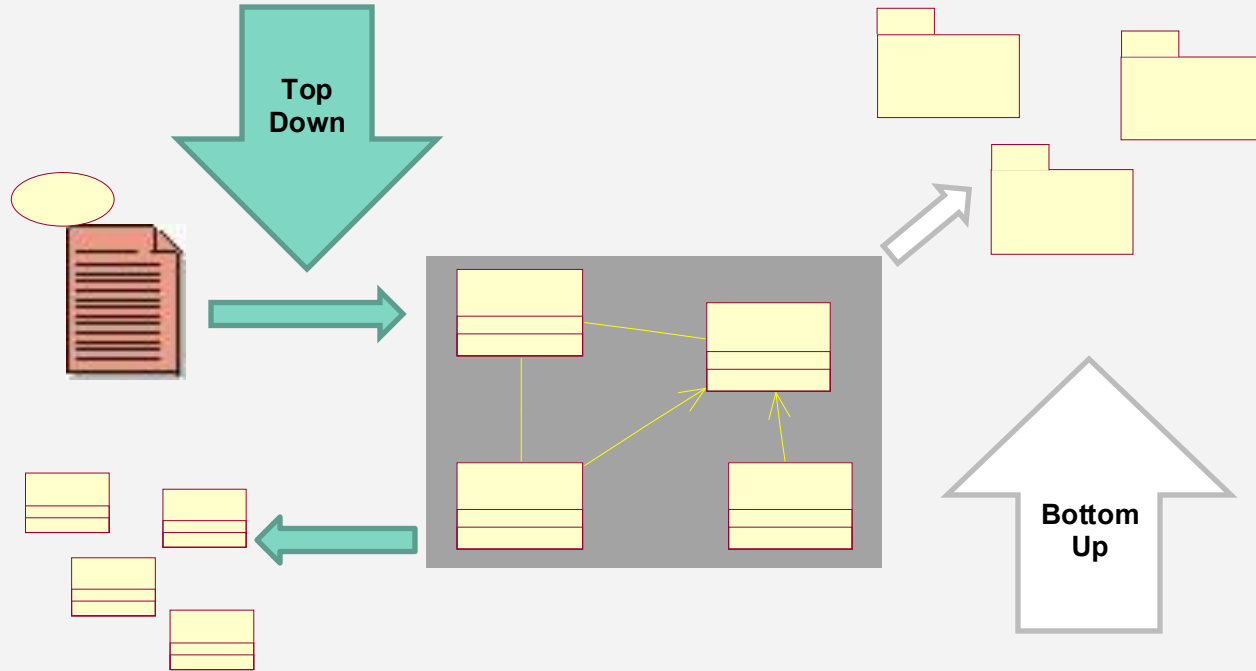
Analiza

vs

Projektowanie

ZROZUMIEĆ PROBLEM	ROZWIĄZAĆ PROBLEM
OGÓLNY PROJEKT	OPERACJE I ATRYBUTY
ZACHOWANIE	EFEKTYWNOŚĆ
STRUKTURA SYSTEMU	BLISKO KODOWANIA
WYMAGANIA FUNKCJONALNE	WYMAGANIA POZAFUNKCJONALNE
MAŁY MODEL 🙄	DUŻY MODEL

TOP-DOWN & BOTTOM-UP



Wzorzec (pattern)

Znane rozwiązanie znanego problemu w określonym kontekście

Fragment rozwiązania, element układanki

Wzorzec architektury: MVC

Wzorzec projektowy: fabryka abstrakcyjna



Szkielet (framework)

Określa ogólne podejście do rozwiązania problemu

Szkielet rozwiązania, którego elementami mogą być np. wzorce projektowe

Ruby on Rails, Spring Framework

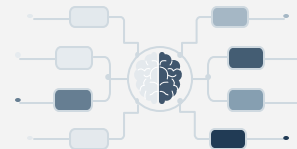


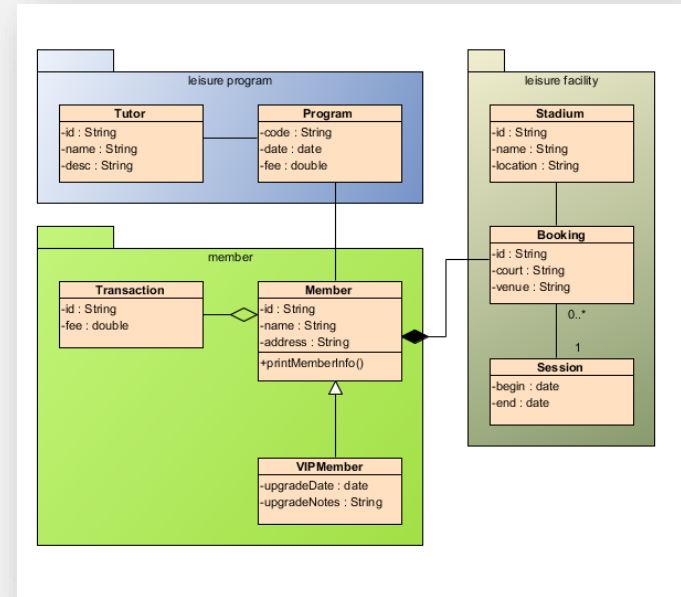
Diagram klas



Podstawowy diagram struktury logicznej



Przedstawia klasy i ich relacje



Model-Code-Deploy Platform

Diagram klas

Klasa

jest abstrakcją; podkreśla pewną charakterystykę i ukrywa inne cechy

Profesor
-id : int = 0 -imie : String -nazwisko : String -dataZatrudnienia : Date +KOD : String
+wystawStopnie(przedmiot : Przedmiot) : boolean +wezUrlop(odDnia : Date, doDnia : Date) : boolean

Obiekt

jest instancją klasy

Kowalski : Profesor
id = 12345 imie = Jan nazwisko = Nowak dataZatrudnienia = 1.IX.2005 KOD = AABCC



Diagram klas: atrybut

Profesor
-id : int = 0 -imie : String -nazwisko : String -dataZatrudnienia : Date +KOD : String
+wystawStopnie(przedmiot : Przedmiot) : boolean +wezUrlop(odDnia : Date, doDnia : Date) : boolean



Nazwana własność klasy, który opisuje zakres wartości jakie może przyjmować instancja własności



Klasa może mieć dowolną liczbę atrybutów lub nie mieć ich wcale

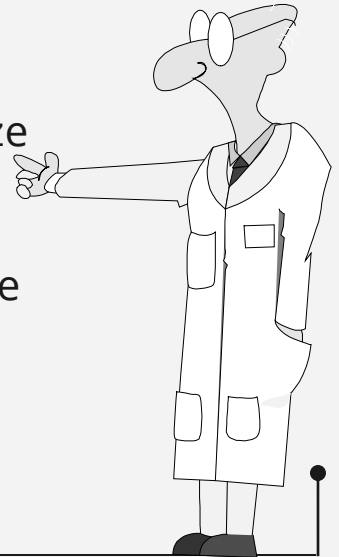


Diagram klas: sygnatura atrybutu

24. X

widoczność	nazwa : typ	[krotność]	{ograniczenia}	= wartość domyślna
+ publiczny	unspecified	1..*	{unique}	
- prywatny	0..1	*	{ordered}	
# chroniony	0..*	od..do	{nonunique}	
~ publiczny w pakiecie	1	2,4..6	{readOnly}	
-	id : int	[1]	{frozen}	= 0

Profesor
-id : int = 0 -imie : String -nazwisko : String -dataZatrudnienia : Date +KOD : String +wystawStopnie(przedmiot : Przedmiot) : boolean +wezUrlop(odDnia : Date, doDnia : Date) : boolean

Diagram klas: operacja

Profesor
-id : int = 0 -imie : String -nazwisko : String -dataZatrudnienia : Date +KOD : String
+wystawStopnie(przedmiot : Przedmiot) : boolean +wezUrlop(odDnia : Date, doDnia : Date) : boolean



Nazwana implementacja pewnej usługi, której wykonania można oczekiwać od każdego obiektu danej klasy



Klasa może mieć dowolną liczbę operacji lub nie mieć ich wcale

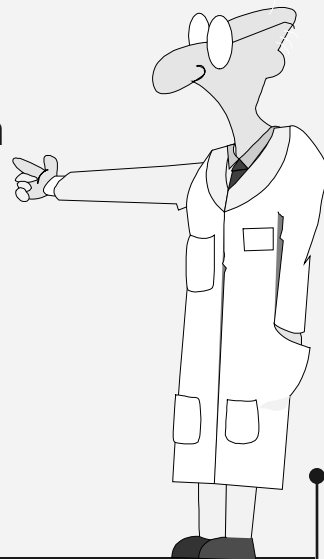


Diagram klas: sygnatura operacji

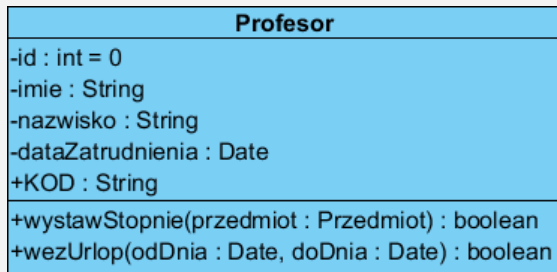
widoczność

nazwa

(parametr_1, .. parametr_n)

: typ

{ograniczenia}



+

wezUrlop

(odDnia, doDnia)

: boolean

{ }

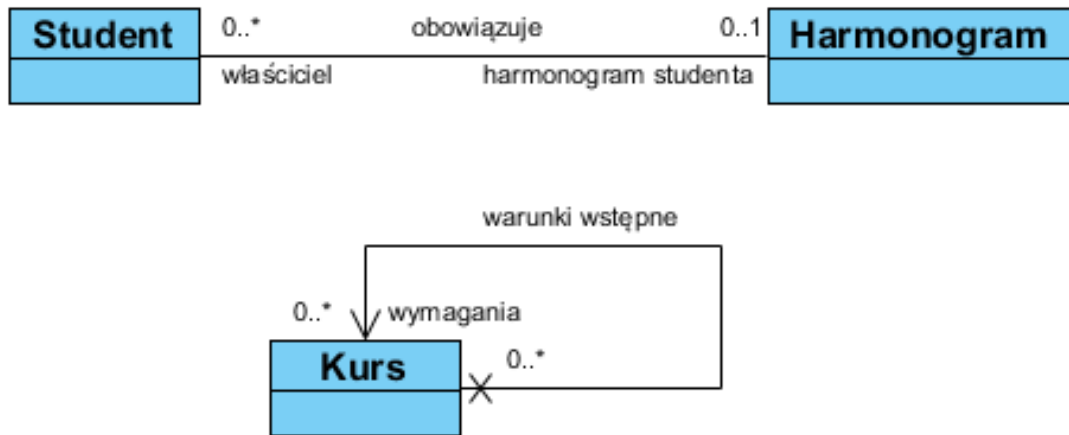
Diagram klas: sygnatura operacji



parametr_1



Relacja: asocjacja



- ✓ **Kierunek**
- ✓ **Krotność**
- ✓ **Rola**

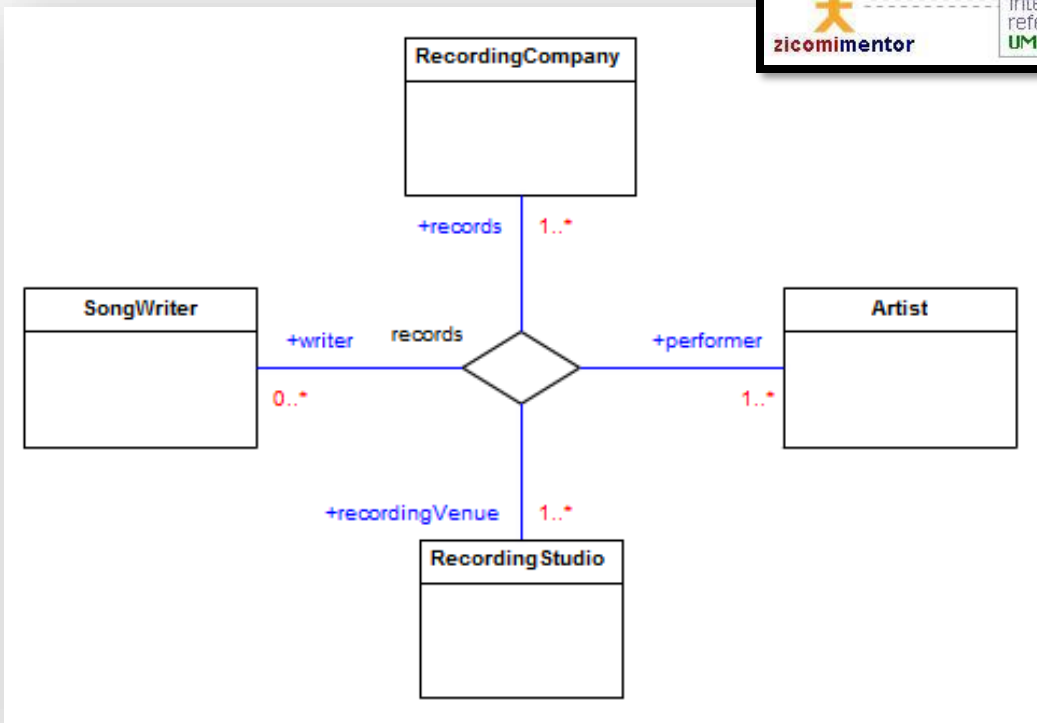


Semantyczna relacja pomiędzy dwoma lub wieloma klasyfikatorami, która reprezentuje zależność między ich instancjami

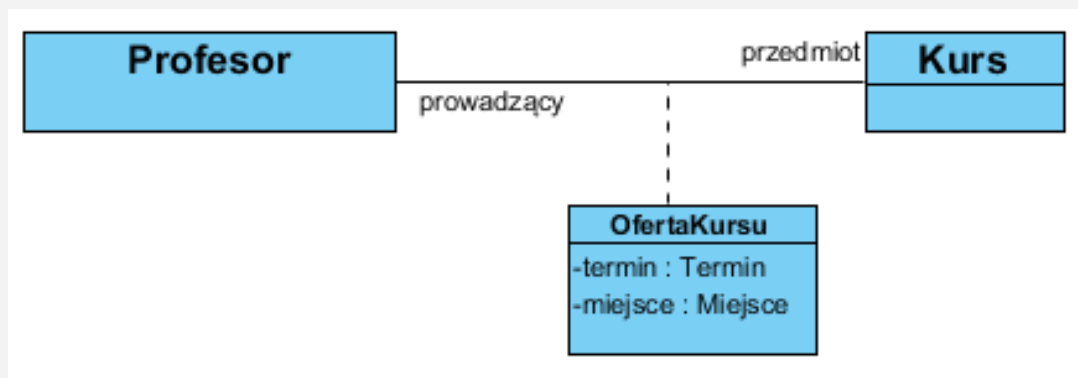


Strukturalna relacja określająca powiązanie pomiędzy obiektami

Relacja: asocjacja

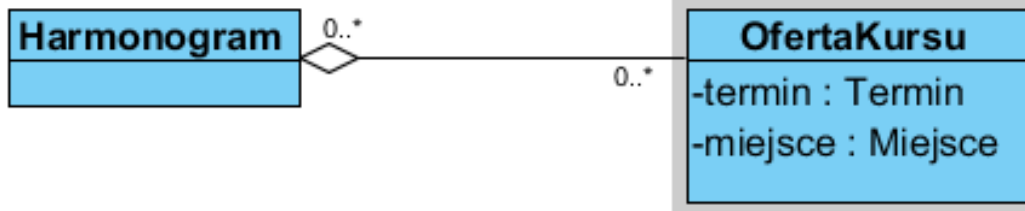


Relacja: asocjacja i klasa asocjacyjna



Opisuje właściwości relacji asocjacyjnej

Relacja: agregacja



Specjalny rodzaj asocjacji modelujący relację całość-część

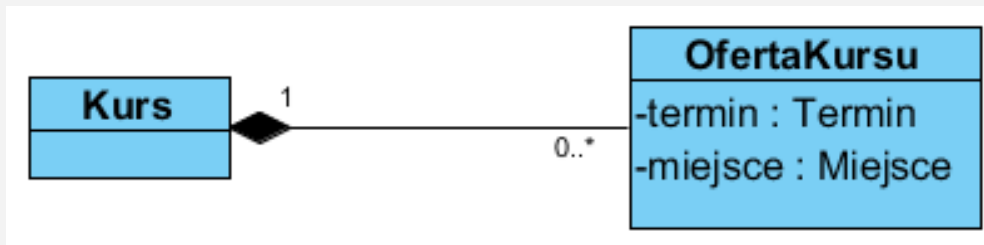


Część może należeć do kilku całości



Całość nie zarządza czasem istnienia części

Relacja: kompozycja



Specjalny rodzaj asocjacji modelujący relację całość-część

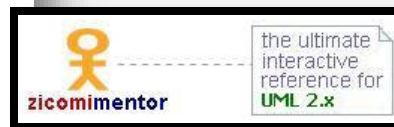
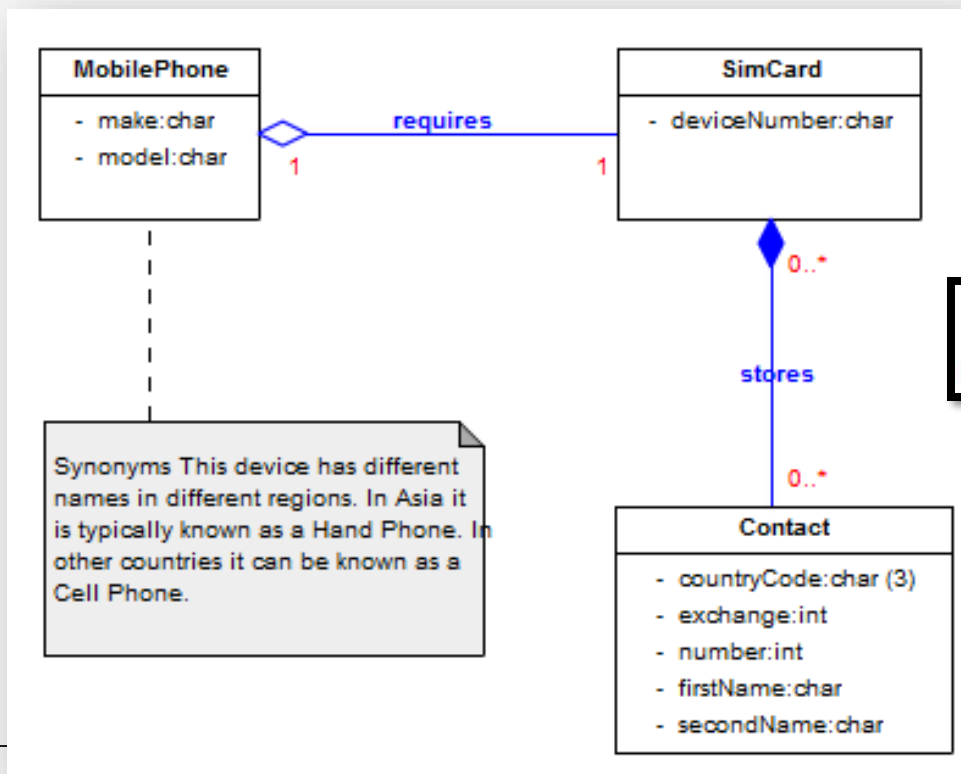


Część może należeć do kilku całości

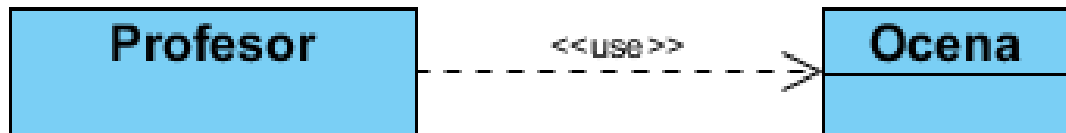


Całość zarządza czasem istnienia części

Relacja: agregacja i kompozycja



Relacja: zależność



Relacja pomiędzy dwoma elementami modelu, gdzie zmiana w jednym elemencie może wymusić zmianę drugiego

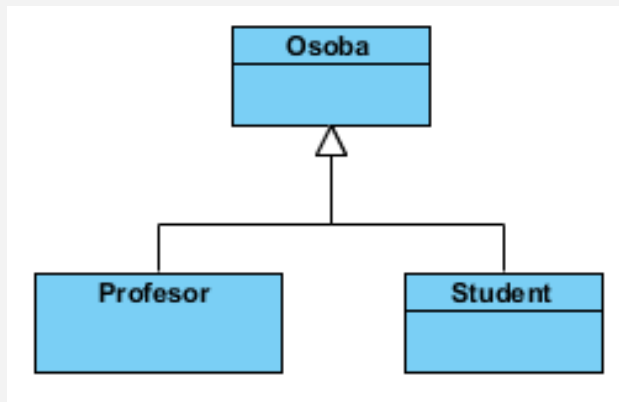


Niestrukuralna relacja: używa



Występuje wtedy, gdy klasa jest parametrem lub zmienną lokalną metody innej klasy

Relacja: generalizacja



Definiuje hierarchię od klas ogólnych do wyspecjalizowanych



Generalizacja, dziedziczenie: jest rodzajem

Relacja: generalizacja



Podklasa dziedziczy od rodzica: atrybuty, operacje i relacje



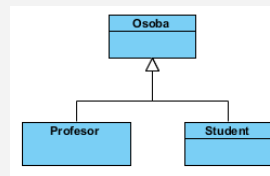
Podklasa może:

Dodać kolejne atrybuty, operacje i relacje

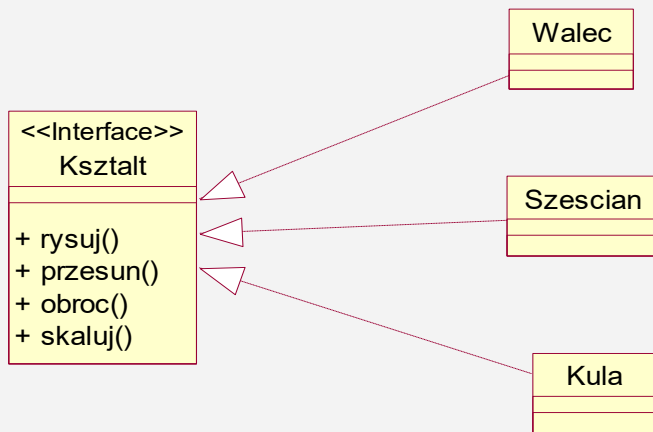
Przedefiniować dziedziczone operacje (ostrożnie)



Wspólne atrybuty, operacje i/lub relacje są umieszczane na najwyższym możliwym poziomie abstrakcji



Relacja: realizacja



Jeden klasyfikator jest zleceniodawcą dla drugiego (definiuje akcje, które drugi wykonuje)



Może wystąpić: interfejsy, przypadki użycia

Klasa abstrakcyjna i interfejs



Nie posiadają obiektów



Klasa abstrakcyjna

- ✓ Deklaruje wspólną funkcjonalność grupy klas
- ✓ Może posiadać implementacje metod



Interfejs

- ✓ Deklaruje funkcjonalność bez implementacji
- ✓ Czysto abstrakcyjny

Stereotyp

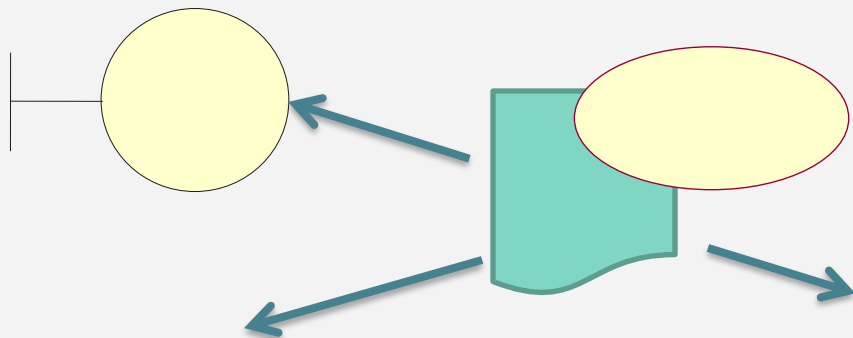
<<stereotyp>>
Klasa

<<Interface>>
Interfejs



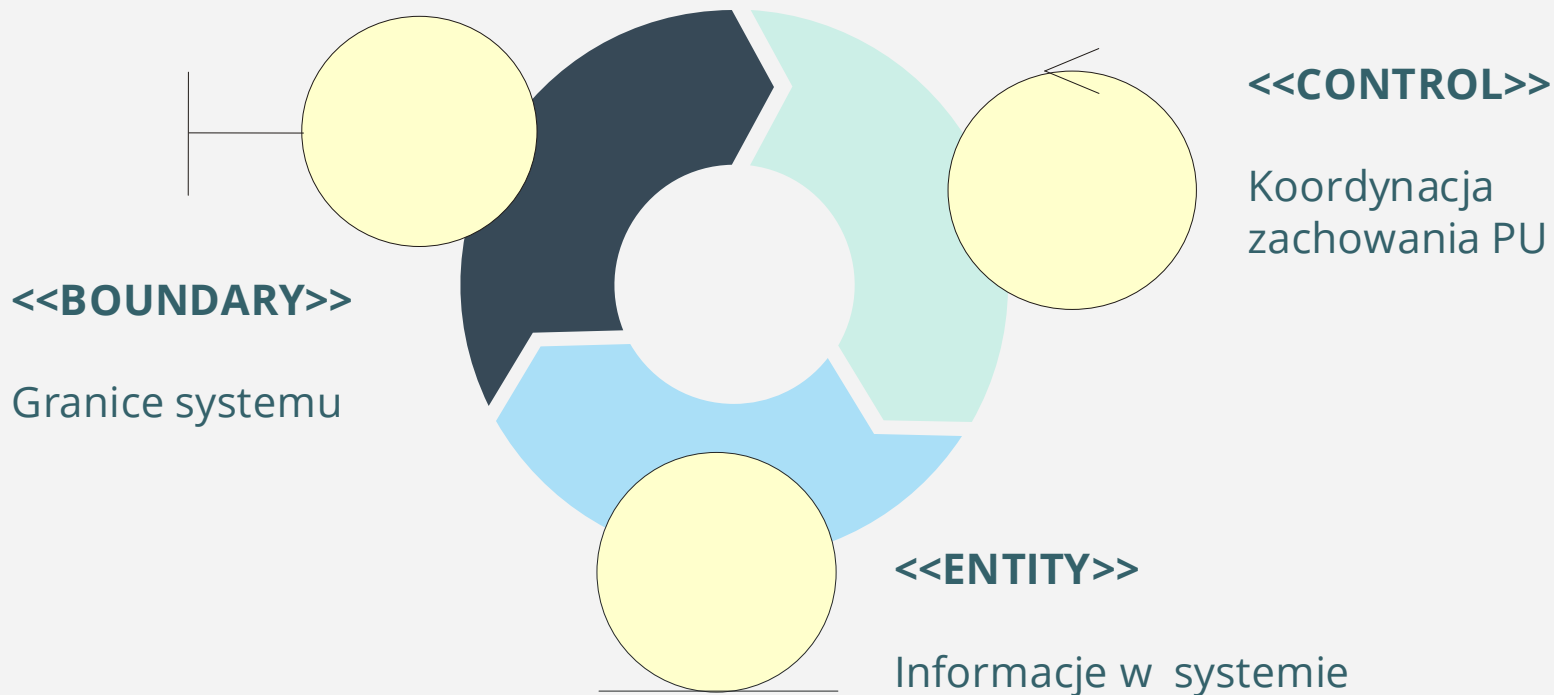
Dodaje pewną semantykę

• PU i klasy analityczne •

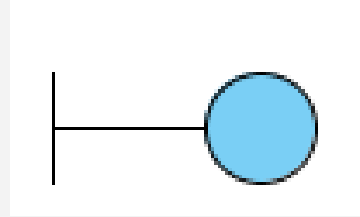


Od modelu PU do projektu

PU i klasy analityczne



Klasa: <<boundary>>



Pośredniczy pomiędzy systemem i otoczeniem



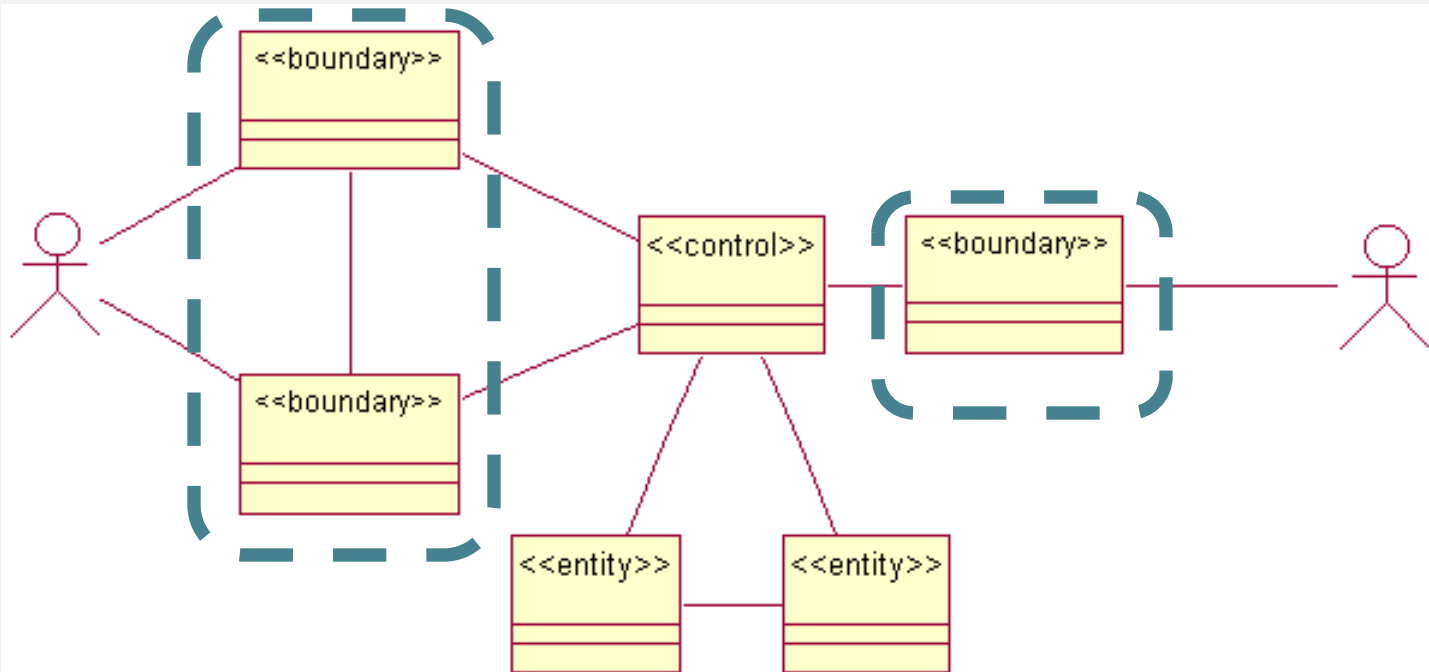
Rodzaje:

- ✓ Klasy interfejsu użytkownika
- ✓ Klasy interfejsu systemu
- ✓ Klasy interfejsu urządzenia



Co najmniej jedna klasa <<boundary>> na parę aktor-przypadek użycia

Klasa: <<boundary>>



Klasa: <<entity>>

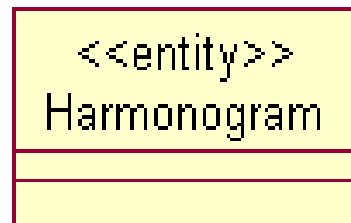
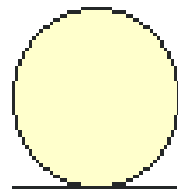
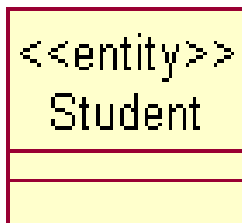


Reprezentuje przechowywane w systemie informacje



Z reguły reprezentuje kluczowe pojęcia związane z systemem, które są w:

- ✓ Słowniku
- ✓ Scenariuszach opisujących PU
- ✓ Kluczowe abstrakcje



OfertaKursu

Klasa: <<control>>



Koordinator zachowania: definiuje logikę kontrolującą i transakcje pomiędzy PU



Jedna klasa typu <<control>> na jeden PU



Niezależna od otoczenia: nie zmienia się jeśli ono się zmienia



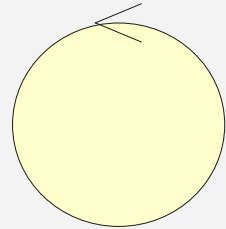
Zmienia się w niewielkim stopniu jeśli zmieni się struktura lub zachowanie klas typu <<entity>>



Używa lub ustala zawartość kilku klas typu <<entity>>



Nie zachowuje się tak samo przy każdej aktywacji



Zachowanie klas: dla każdego scenariusza



Identyfikujemy klasy

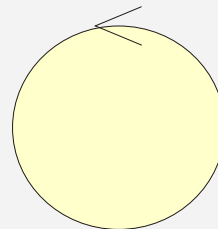


Rozdzielamy pomiędzy nie obowiązki



Modelujemy pomiędzy nimi interakcje

- ✓ Diagram sekwencji
- ✓ Diagram komunikacji
- ✓ Diagram przeglądu interakcji
- ✓ Diagram uwarunkowań czasowych



Zachowanie klas: jak rozdzielić obowiązki



<<boundary>>: komunikacja z aktorem



<<entity>>: operacje na heremetyzowanych danych



<<control>>: zachowanie konkretnego scenariusza

Zachowanie klas: jak rozdzielić obowiązki



Jedna klasa ma dane to ona ma obowiązki 😊



Wiele klas operuje na danych:

- ✓ Obowiązki ma jedna klasa i dodajemy jej relacje z pozostałymi
- ✓ Tworzymy nową klasę i j.w.
- ✓ Obowiązki dajemy klasie typu <<control>> i j.w.

Zachowanie : diagram sekwencji



Prezentuje interakcje poprzez wysyłanie wiadomości



Dwa wymiary:

- ✓ Czas w dół
- ✓ Różne obiekty w poziomie



Składniki:

- ✓ Obiekty
- ✓ Przepływy
- ✓ Opisy

Zachowanie : diagram sekwencji



Prezentuje interakcje poprzez wysyłanie wiadomości



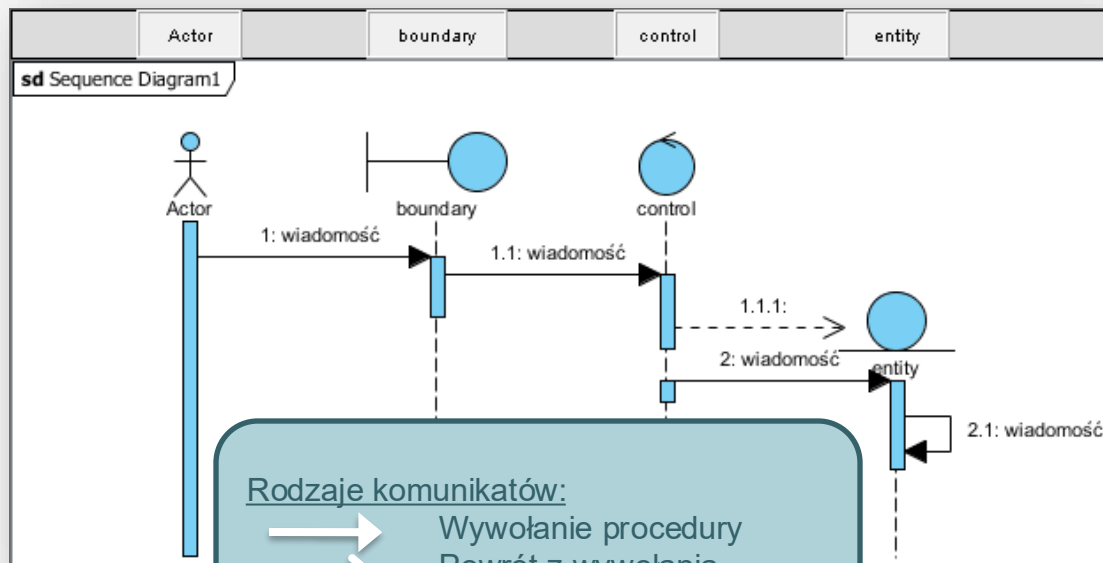
Dwa wymiary:

- ✓ Czas
- ✓ Różne obiekty



Składniki:

- ✓ Obiekty
- ✓ Przepływy
- ✓ Opisy

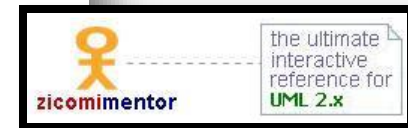
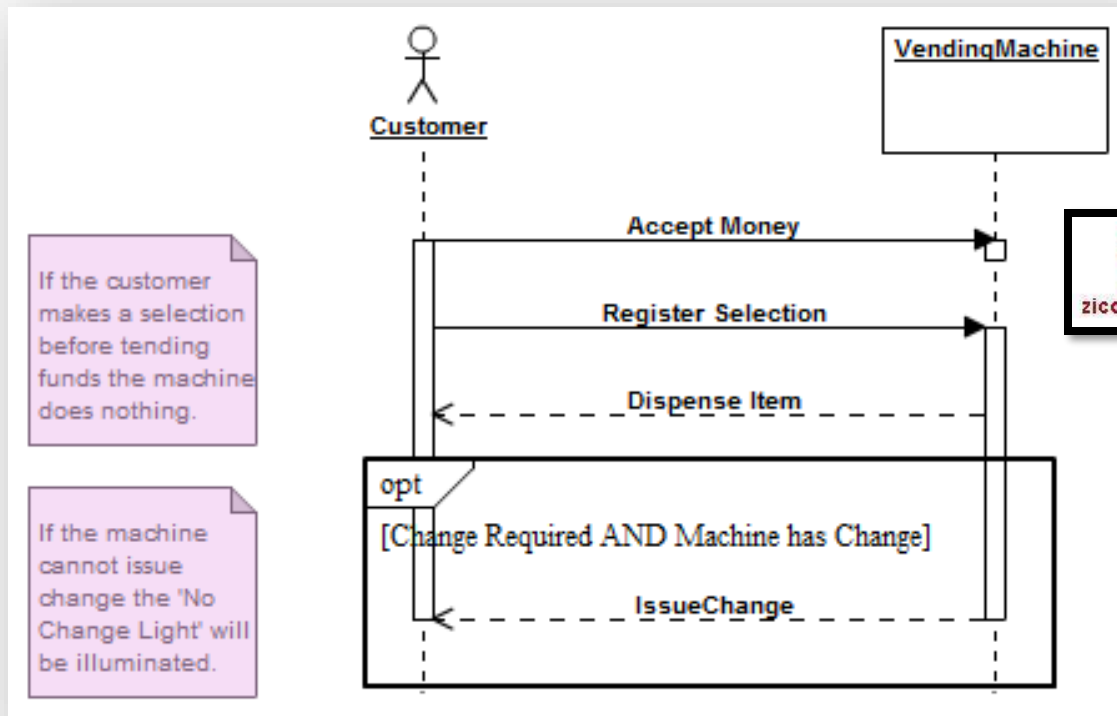


Rodzaje komunikatów:



Wywołanie procedury
Powrót z wywołania
Wywołanie asynchroniczne

Zachowanie : diagram sekwencji



Zachowanie : diagram komunikacji



Podobna funkcja jak diagram sekwencji



Interakcje zgromadzone wokół ról i ich powiązań



Czas pokazany jedynie poprzez numerowanie

Zachowanie : diagram komunikacji

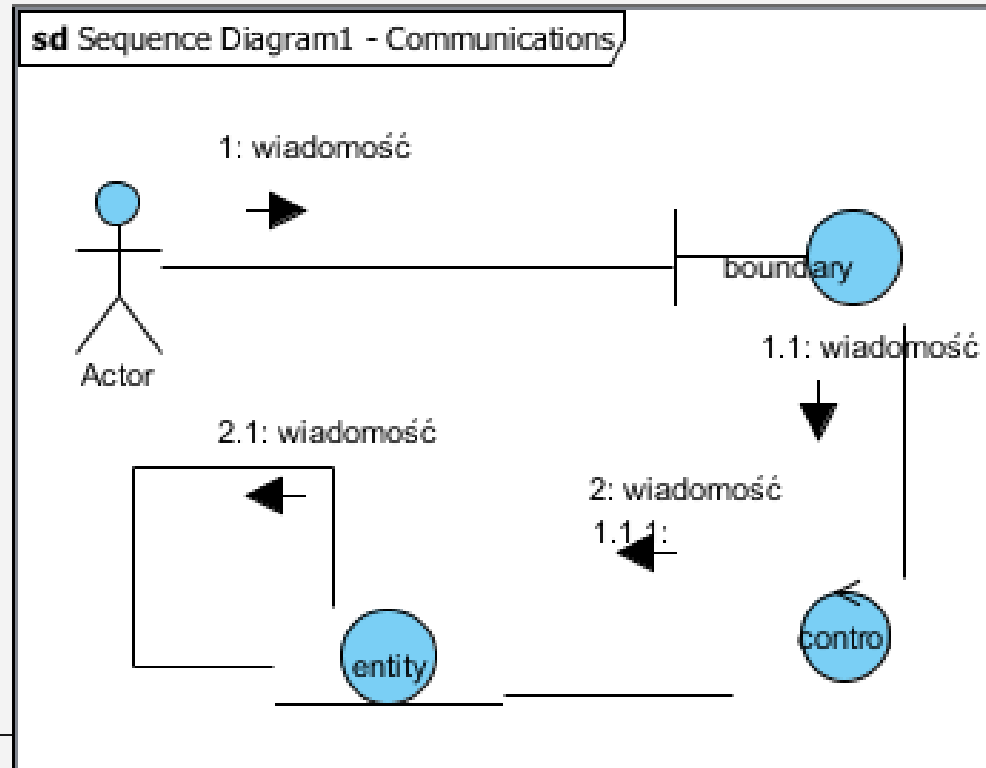


Diagram komunikacji Diagram sekwencji

**POKAZUJE RELACJE WRAZ Z
INTERAKCJAMI**

**POKAZUJE DOKŁADNĄ SEKWENCJĘ
WYMIANY KOMUNIKATÓW**

**LEPSZY DO WIZUALIZACJI
ELEMENTÓW BIORĄCYCH UDZIAŁ**

**LEPSZY DO WIZUALIZACJI
PRZEBIEGU AKCJI**

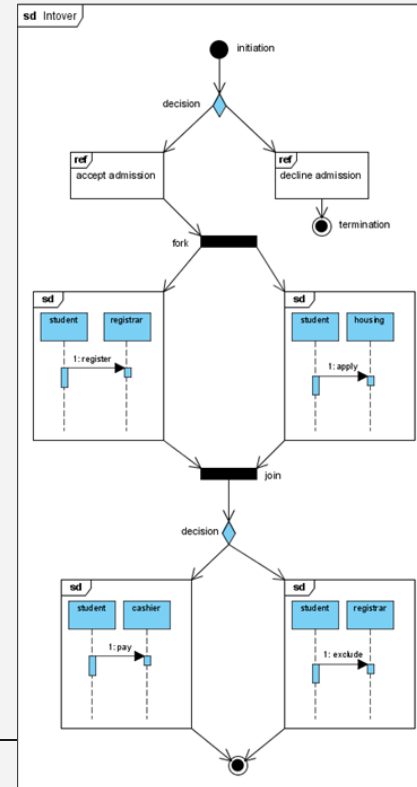
Zachowanie : diagram przeglądu interakcji



Diagram czynności, w którym węzłami są diagramy



Przedstawia przepływ sterowania



Zachowanie : diagram maszyny stanów



Reprezentuje zachowanie obiektu wzbudzone zdarzeniami

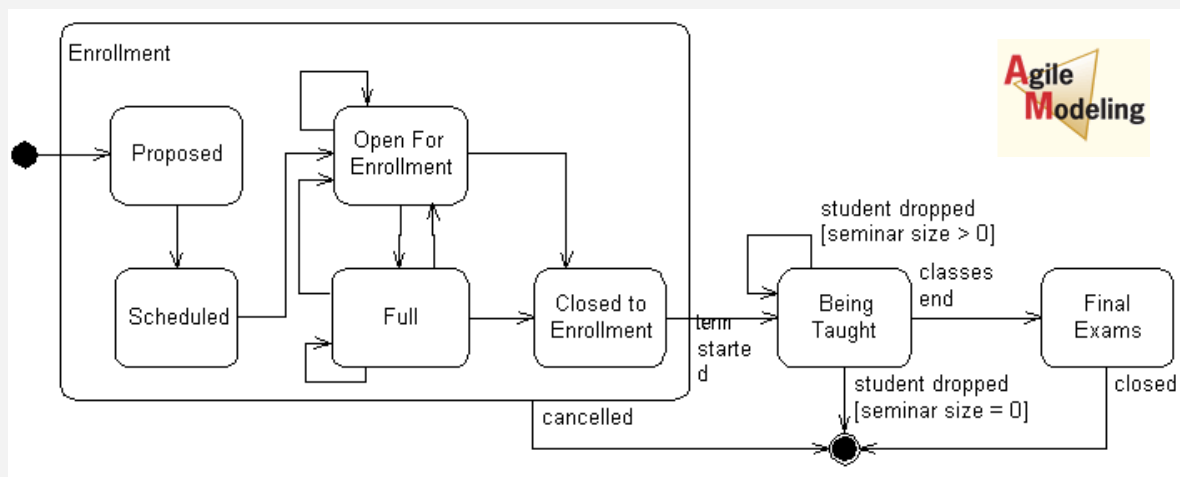


Opisuje możliwe sekwencje stanów i akcji przez które przechodzi element w czasie swego istnienia pod wpływem zdarzeń



Obiektami są zwykle klasy, lecz mogą też być aktorzy, metody, operacje itp...

Zachowanie : diagram maszyny stanów



UML dzisiaj



Głównie w uproszczonej, pragmatycznej formie



Stosuje się go **wybiórczo**: głównie diagramy przypadków użycia, klas sekwencji,



Coraz częściej zastępuje go **C4 Model** (Context-Container-Component-Code),



Diagramy tworzone automatycznie z kodu: Visual Paradigm, PlantUML, StarUML

AI & OOAD



AI wspiera inżynierów:

- ✓ Analizuje kod i generuje diagramy UML
- ✓ Wspiera tworzenie scenariuszy użycia
- ✓ Sugeruje refaktoryzację



W nowoczesnych środowiskach:

- ✓ diagramy UML i C4 generowane automatycznie podczas buildów
- ✓ AI analizuje zmiany w kodzie i aktualizuje modele architektury
- ✓ raporty UML mogą być publikowane w Confluence lub repozytoriach Git

Kod źródłowy → AI analiza → Diagram UML → Sugestie refaktoryzacji → CI/CD



Koniec!